Manual Técnico Definitivo: Arquitectura, Seguridad y Fundamentos de Payé Base

> **Proyecto**: Payé Base (Plantilla Base Single-Tenant)
> **Plataforma**: ASP.NET Core MVC (.NET 10 LTS) & PostgreSQL 17
> **Entorno objetivo**: Aplicaciones a medida para comercios y pequeñas empresas
> **Directorio base**: `d:\Desarrollo\Paye_POS`

1. Manifiesto Arquitectónico: Qué es Payé Base y Por Qué ASP.NET Core Moderno

Payé Base es la plantilla base de grado profesional de Payé Digital para construir software de gestión (TPV, CRM, Facturación, Gestión Operativa) adaptado a cada cliente. Se concibió para erradicar dos grandes plagas del desarrollo web contemporáneo:

1. **La sobreingeniería frágil**: microservicios prematuros, bibliotecas de terceros que duplican el framework (MediatR, AutoMapper), capas de abstracción inútiles (repositorios genéricos que envuelven EF Core) y configuraciones multi-inquilino en tablas que aumentan exponencialmente el riesgo de fuga de datos fiscales.
2. **La negligencia de seguridad**: tokens JWT guardados en `localStorage` (vulnerables a robo por XSS), permisos basados en strings fijos de rol, contraseñas de demostración fijas en código y falta de control estricto en el arranque.

Por Qué ASP.NET Core Moderno (.NET 10 LTS) es una Bestia de Ingeniería

Quienes aprendieron ASP o ASP.NET en los años 90 o 2000 recuerdan conceptos obsoletos: *WebForms*, el infame `ViewState`, mezclar código en archivos `.aspx.cs`, IIS pesado y atado a Windows, y un modelo de peticiones síncrono.

El ASP.NET Core actual no tiene nada que ver con aquello. Es un runtime reescrito desde cero, multiplataforma, modular y de ultra alto rendimiento:

- **Kestrel**: Servidor HTTP asíncrono basado en sockets y `System.IO.Pipelines`, capaz de procesar millones de peticiones por segundo con asignación casi nula de memoria en el Garbage Collector.
- **Pipeline de Middleware Unificado**: Cada petición atraviesa una tubería bidireccional limpia y secuencial configurada en código (`Program.cs`), donde cada paso puede inspeccionar, modificar o cortocircuitar la solicitud.
- **Contenedor de Inversión de Control (IoC) Integrado**: Inyección de dependencias nativa de primer nivel sin necesidad de contenedores externos.
- **Cero dependencias de IIS o Windows**: Se compila directamente a imágenes Docker ultraligeras basadas en Alpine o Debian Chiseled (`mcr.microsoft.com/dotnet/aspnet:10.0`).
- **Seguridad Criptográfica Integrada**: Implementaciones oficiales de hashing de contraseñas (PBKDF2 con sal criptográfica), tokens de un solo uso criptográficos, protección de datos y validación de seguridad de transporte.

2. Lo que Hace que ASP.NET Core Sea lo que Es: Anatomía del Runtime

Para entender por qué Payé Base es robusta, es indispensable entender los pilares internos sobre los que opera ASP.NET Core:

2.1. Kestrel y la Eliminación de Fugas de Información

En `Program.cs`, el servidor web Kestrel se endurece inmediatamente:

```
csharp
builder.WebHost.ConfigureKestrel(serverOptions => serverOptions.AddServerHeader = false);
```

Esto elimina la cabecera `Server: Kestrel`, impidiendo que escáneres automatizados identifiquen la tecnología o versión exacta del host.

2.2. Inyección de Dependencias (IoC) y Ciclos de Vida

El contenedor nativo gestiona tres ciclos de vida esenciales:

- **Transient (`AddTransient`)**: Una instancia nueva cada vez que se solicita.
- **Scoped (`AddScoped`)**: Una única instancia por petición HTTP. Es el ciclo obligatorio para `PayeDbContext`, `ICurrentUserService`, `IAuditLogger` y servicios de negocio, asegurando que toda una transacción HTTP comparta la misma sesión de base de datos.

- **Singleton (`AddSingleton`)**: Una única instancia durante toda la vida del proceso. Se usa para componentes inmutables o sin estado como `IPermissionDefinitionProvider`, `SecurityHeadersMiddleware`, `IPdfInvoiceGenerator` y `IAuthorizationPolicyProvider`.

2.3. Patrón de Opciones y Fail-Closed en Arranque (`ValidateOnStart`)

ASP.NET Core permite enlazar secciones de configuración (`appsettings.json`, variables de entorno) a clases fuertemente tipadas. Payé Base añade **`ValidateOnStart`** obligatorio:

```
csharp
builder.Services
    .AddOptions<DatabaseSettings>()
    .Bind(builder.Configuration.GetSection(DatabaseSettings.SectionName))
    .ValidateDataAnnotations()
    .ValidateOnStart();
```

Si alguna variable requerida (cadena de conexión a base de datos, directorio de claves, nombre del negocio) no existe o contiene el marcador `"CHANGE_ME"`, **el servidor se niega a arrancar**. La aplicación falla inmediatamente en el despliegue antes de exponer un solo puerto HTTP desconfigurado.

2.4. Data Protection API (`IDataProtectionProvider`)

ASP.NET Core incluye un subsistema de cifrado simétrico y autenticación criptográfica que protege:

- Las cookies de sesión y autenticación.
- Los tokens antiforgery (CSRF).
- Los tokens de restablecimiento de contraseña y confirmación de correo.
- Secretos de negocio (como contraseñas y claves técnicas en el CRM).

En Payé Base, las claves criptográficas se aíslan y persisten fuera del contenedor Docker en un volumen seguro (`/var/lib/paye/keys`), garantizando que reiniciar el contenedor no desautentique a los usuarios ni invalide sesiones.

3. Estructura de la Solución (Clean Architecture Pragmática)

La solución se divide en tres proyectos de código productivo y un proyecto de pruebas rigurosas:

```
d:\Desarrollo\Paye_POS\
├── src\
│   ├── Paye.Core\              # Dominio puro y contratos (Cero dependencias de EF o MVC)
│   ├── Paye.Infrastructure\    # EF Core, PostgreSQL, Identity, Archivos, Auditoría
│   └── Paye.Web\               # Controladores, Vistas, SPA Bridge, Middleware, Program.cs
├── tests\
│   └── Paye.Tests\             # Tests con Testcontainers PostgreSQL y TestDatabaseGuard
├── docs\                       # Documentación viva de arquitectura y seguridad
├── Directory.Build.props       # Configuración global (.NET 10, C# 13, Nullable,
WarningsAsErrors)
├── docker-compose.yml          # Topología de producción (App + PostgreSQL 17)
└── Dockerfile                  # Construcción multi-etapa segura y no root
```

Matriz de Responsabilidades por Capa

Capa	Lo que CONTIENE	Lo que TIENE PROHIBIDO
`Paye.Core`	Entidades de dominio, contratos de servicios (`I*Service`), definiciones de permisos (`IPermissionDefinitionProvider`), objetos de valor (`Money`, `VatRate`), modelos de configuración (`*Settings`).	Referenciar EF Core, Npgsql, ASP.NET Core MVC, HttpContext o librerías externas.
`Pave.Infrastructure`	`PayeDbContext`, configuraciones de entidades (`IEntityTypeConfiguration`), migraciones de base de datos, servicios de archivos en disco, siembra de Identity, auditoría en BD.	Referenciar controladores MVC, ViewModels o vistas Razor.
`Paye.Web`	`Program.cs`, middleware HTTP, controladores (`AccountController`, etc.), ViewModels, vistas Razor, componentes de vista (`ViewComponents`), estáticos (`wwwroot`), frontend SPA compilado.	Ejecutar consultas SQL directas o interactuar con el sistema de archivos sin pasar por los contratos de `Core`.
`Pave.Tests`	Pruebas de integración reales contra contenedores efímeros de PostgreSQL con Testcontainers.	Ejecutar tests contra bases de datos que coincidan con `ConnectionStrings__DefaultConnection`.

4. Seguridad por Defecto (Innegociable y Exhaustiva)

Cada una de las 14 reglas de seguridad de Payé Base se implementa a nivel de framework de manera estricta y verificable:

4.1. Deny-by-Default Global (`FallbackPolicy`)

En la configuración de autorización en `Program.cs`:

```
csharp
builder.Services.AddAuthorization(options =>
{
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
});
```

> IMPORTANT
>
> Un desarrollador no necesita recordar poner `[Authorize]` en cada controlador. **Cualquier endpoint nuevo exige login automáticamente**. La única forma de que una ruta sea pública es decorarla intencionadamente con `[AllowAnonymous]` (restringido a `/health`, login y recuperación).

4.2. Autenticación con ASP.NET Core Identity Endurecida

Payé Base prohíbe programar algoritmos caseros de autenticación. Utiliza el motor oficial de Identity sobre PostgreSQL con políticas de máxima seguridad:

- Longitud mínima de contraseña: **12 caracteres** con mayúsculas, minúsculas, dígitos y símbolos.
- Bloqueo por fuerza bruta (**Lockout**): Tras **5 intentos fallidos**, la cuenta se bloquea automáticamente durante **15 minutos**.
- Soporte nativo para doble factor de autenticación (**2FA TOTP**) compatible con Google Authenticator / Apple Passwords mediante algoritmo RFC 6238.

4.3. Invalidación Instantánea de Sesiones (`SecurityStampValidator`)

En arquitecturas basadas en JWT, revocar un token antes de su caducidad es complejo y requiere listas negras en memoria. En Payé Base:

```
csharp
builder.Services.Configure<SecurityStampValidatorOptions>(options =>
{
    options.ValidationInterval = TimeSpan.Zero;
});
```

Cada usuario posee un `SecurityStamp` en la base de datos. Si un administrador desactiva a un usuario (`IsActive = false`), cambia sus roles o el usuario resetea su contraseña, el sello de seguridad cambia. **En la siguiente petición HTTP exacta**, el middleware detecta el desfase y expulsa al usuario al login inmediatamente.

4.4. Autorización Granular por Permisos Dinámicos

Acoplar código a nombres de roles (`[Authorize(Roles = "Encargado")]`) es un antipatrón que rompe la modularidad cuando un negocio quiere personalizar qué puede hacer un empleado. Payé Base utiliza **Claims de Permiso**:

```
csharp
[Authorize(Policy = FacturacionPermissions.Crear)]
public async Task<IActionResult> CreateInvoice(...)
```

1. **Definición**: Cada módulo define sus permisos implementando `IPermissionDefinitionProvider`.
2. **Construcción Dinámica**: `PermissionPolicyProvider` intercepta cualquier política con nombre `Modulo.Accion` y la crea al vuelo.
3. **Manejador de Autorización**: `PermissionAuthorizationHandler` comprueba si el usuario posee el claim `paye.permission` asignado a través de sus roles en la tabla `AspNetRoleClaims`.
4. **Protección del Último Propietario**: La lógica de negocio impide desactivar o degradar al último usuario con rol `Propietario` del sistema, evitando bloqueos accidentales de administración.

4.5. Cabeceras de Seguridad y CSP Estricta

El middleware `SecurityHeadersMiddleware` inyecta en cada respuesta HTTP:

```
http
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src
'self' data:; font-src 'self'; connect-src 'self'; frame-ancestors 'none'; base-uri 'self';
form-action 'self'
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

> NOTE

> La política CSP **no contiene** `unsafe-inline` **ni** `unsafe-eval`. No se permite `<script>` ni `<style>` embebidos en el HTML ni llamadas a `eval()`, erradicando la superficie de ataque XSS. Todo el código JavaScript y CSS reside en archivos estáticos compilados en `/wwwroot`.

4.6. Protección Global contra CSRF (Antiforgery)

- **Validación Automática en Escrituras**:

  ```
  csharp
  builder.Services.AddControllersWithViews(options =>
  {
      options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
  });
  ```

  Toda petición HTTP `POST`, `PUT`, `DELETE` o `PATCH` exige automáticamente un token antiforgery válido.
- **Sincronización con el Frontend / SPA**: `_Layout.cshtml` genera un token antiforgery en una etiqueta `<meta name="csrf-token" content="..." />` y en un input oculto. El cliente React/fetch lee dicho token y lo envía en la cabecera `RequestVerificationToken`.

4.7. Rate Limiting Nativo

Utilizando `System.Threading.RateLimiting` (.NET nativo):

```
csharp
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.AddPolicy("auth-strict", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 10,
                Window = TimeSpan.FromMinutes(1)
            }));
});
```

Los endpoints de autenticación (`/account/login`, `/account/demo-login`) están protegidos contra ataques de fuerza bruta por IP con respuesta HTTP 429 estandarizada.

4.8. Subida de Archivos Privados con Validación de Magic Bytes

Los archivos subidos por usuarios (documentos fiscales, adjuntos) nunca se almacenan en `wwwroot` (donde un servidor web podría ejecutarlos estáticamente).

El servicio `LocalPrivateFileStorage`:

1. Almacena en `Security:PrivateUploadsPath` (directorio fuera de la raíz web).
2. Asigna un nombre aleatorio UUID (`Guid.NewGuid()`), ignorando el nombre de archivo enviado por el cliente para evitar *Path Traversal*.
3. **Verifica las firmas binarias reales (*Magic Bytes*)**:
   - PDF: `%PDF` (`0x25 0x50 0x44 0x46`)
   - PNG: `0x89 0x50 0x4E 0x47`
   - JPEG: `0xFF 0xD8 0xFF`
   - WEBP: `RIFF....WEBP` Si alguien renombra un script `.php` o `.exe` a `.pdf`, el validador binario lo rechaza instantáneamente.
4. Para descargarlo, se accede a través de `PrivateFilesController` con verificación de permisos, sirviéndose con cabeceras `Content-Disposition: attachment` y `X-Content-Type-Options: nosniff`.

4.9. El Sandbox Demo Aislado (`DemoSandbox`)

En lugar de crear usuarios de prueba con contraseñas fijas (como `admin / admin123`, prohibido taxativamente):

- Está **desactivado por defecto** (`Fail-closed`).
- Al activarse en local/demos (`DemoSandbox__Enabled=true`), el botón "*Entrar al Modo Demo Aislado*" crea una cuenta efímera `demo-<guid>` con una contraseña aleatoria de 32 bytes criptográficos que se descarta en memoria.
- Asigna únicamente permisos aislados de módulo (`IDemoModuleDataProvider`), bloqueando explícitamente administración de usuarios, roles o subida de archivos físicos.
- Aísla los registros para que solo esa sesión vea sus propios datos de prueba.
- **Purga total**: Al cerrar sesión, se eliminan físicamente de PostgreSQL todos los registros creados, la traza de auditoría y la propia cuenta efímera.

5. Arquitectura Modular Desenchufable (`Modules/<NombreModulo>/`)

Payé Base organiza las capacidades de negocio en módulos autocontenidos en compilación. La regla de oro es: **desenchufar un módulo debe reducirse a borrar sus carpetas y eliminar una sola línea en `Program.cs` (`builder.Services.Add<Modulo>Module()`)**.

5.1. Proveedores de Extensión Modular

Un módulo no toca el núcleo para registrarse; implementa contratos abiertos:

1. **`IPermissionDefinitionProvider`**: Declara los permisos del módulo (p. ej. `Facturacion.Ver`, `Facturacion.Crear`) y qué roles base los reciben.
2. **`IModuleNavigationProvider`**: Proporciona los enlaces de navegación del módulo con su icono SVG, ruta y permiso requerido para que el menú lateral (`SidebarNavigationViewComponent`) los pinte automáticamente solo si el usuario tiene permiso.
3. **`IDemoModuleDataProvider`**: Permite al módulo sembrar datos de muestra al entrar al sandbox demo y limpiarlos al salir.
4. **`IEntityTypeConfiguration<T>`**: Cada entidad del módulo define sus índices y claves en su propia clase. `PayeDbContext` las descubre en lote con:

   ```
   csharp
   modelBuilder.ApplyConfigurationsFromAssembly(typeof(PayeDbContext).Assembly);
   ```

5.2. Módulo de Facturación y Cumplimiento Fiscal (VeriFactu / Ley Antifraude)

Como ejemplo insignia, el módulo `Facturacion` implementa los requisitos exigidos por la normativa tributaria española (RD 1007/2023):

- **Inmutabilidad de Facturas Emitidas**: Una factura en estado `Emitida` no puede modificarse ni borrarse (garantizado en base de datos y en servicios).
- **Facturas Rectificativas**: Las correcciones se realizan exclusivamente mediante documentos de tipo `Rectificativa` que referencian a la factura original.
- **Encadenamiento Criptográfico SHA-256**: Cada registro de facturación calcula su huella digital combinando el NIF emisor, número de factura, fecha, importe y el hash del registro anterior, garantizando la trazabilidad no manipulable.
- **Series Contables Separadas**: Numeración correlativa estricta por año fiscal sin huecos.

6. Persistencia, Concurrencia y Manejo de Datos (PostgreSQL + EF Core)

6.1. Concurrencia Optimista Nativa con `xmin`

Para evitar que dos usuarios sobreescriban los datos del otro simultáneamente sin darse cuenta (*lost updates*), Payé Base aprovecha la columna oculta de PostgreSQL **`xmin`** (el identificador de transacción que modifica la fila):

```
csharp
builder.Property(e => e.RowVersion)
    .IsRowVersion();
```

En PostgreSQL, esto se mapea a la columna de sistema `xmin`. Cuando se realiza un `UPDATE`, EF Core genera:

```
sql
UPDATE "clientes" SET "nombre" = @p0, ...
WHERE "id" = @p1 AND "xmin" = @p2;
```

Si otro usuario modificó la fila en milisegundos intermedios, el `xmin` habrá cambiado, la consulta afectará a 0 filas y EF Core lanzará una excepción `DbUpdateConcurrencyException`. El servicio la captura y retorna un error de conflicto limpio al usuario, impidiendo que destruya los cambios ajenos.

6.2. Interceptor de Auditoría Automática

En lugar de rellenar manualmente quién creó o modificó cada entidad: `AuditableEntitySaveChangesInterceptor` inspecciona en cada llamada a `SaveChangesAsync()` todas las entidades que heredan de `AuditableEntity`. De forma transparente:

- Extrae el ID del usuario del `ICurrentUserService`.
- Asigna `CreatedAtUtc` y `CreatedBy` en inserciones.
- Asigna `UpdatedAtUtc` y `UpdatedBy` en actualizaciones.

6.3. Borrado Lógico con Filtros Globales de Consulta

Toda entidad crítica implementa `ISoftDeletable`. En `PayeDbContext`:

```
csharp
modelBuilder.Entity<Cliente>().HasQueryFilter(e => !e.IsDeleted);
```

Cualquier consulta LINQ en el sistema añade automáticamente `WHERE "is_deleted" = FALSE`. Para purgas físicas explícitas (como en el modo demo), se utiliza `.IgnoreQueryFilters()`.

6.4. Fechas en UTC e Importes con Precisión Bancaria

- **Fechas**: Se guardan estrictamente en UTC (`timestamptz`). Al presentarlas al usuario o acotar el día fiscal se convierten a `Europe/Madrid`.
- **Importes**: Prohibido usar `float` o `double`. Se usa siempre `decimal` / `numeric(18,2)` con tipos `Money` y redondeo bancario de número par (`MidpointRounding.ToEven`).

7. El Puente Híbrido: SPA Moderna (React + Vite) sobre ASP.NET Core

Payé Base une lo mejor de dos mundos: la solidez y seguridad de sesión de ASP.NET Core con la fluidez y reactividad de una SPA moderna.

Por Qué Payé Base NO Usa Tokens en `localStorage`

En muchas arquitecturas SPA desacopladas, se devuelve un token JWT y se guarda en `localStorage`. **Eso es un grave error de seguridad**: cualquier vulnerabilidad XSS (o dependencia de npm comprometida) puede leer `localStorage` y exfiltrar el token del usuario.

En Payé Base:

1. La sesión se guarda en una cookie `HttpOnly`, inaccesible para cualquier código JavaScript.
2. La página sirve `_Layout.cshtml` con un contenedor `<div id="root">`.
3. El paquete compilado de la SPA (`paye-app.js`) se monta en `#root`.
4. Todas las llamadas a las APIs REST del servidor se realizan con `credentials: "same-origin"`. El navegador adjunta la cookie de sesión automáticamente sin que JavaScript toque nunca una credencial.
5. Los ataques CSRF quedan completamente neutralizados por el token de verificación que la SPA envía en la cabecera HTTP de cada petición mutativa.

8. Guía de Operaciones, Despliegue y Pruebas Automatizadas

8.1. Contenedor Docker de Producción

El `Dockerfile` utiliza construcción multi-etapa (.NET SDK 10 en build, ASP.NET Runtime 10 en final):

- Ejecución bajo el usuario no root del sistema (`USER $APP_UID`), impidiendo que un atacante comprometa el host si vulnera el proceso.
- Directorios persistentes con permisos correctos: `/var/lib/paye/keys` (claves de cifrado) y `/var/lib/paye/private-uploads` (archivos privados).
- Healthcheck integrado en el propio contenedor: `dotnet Paye.Web.dll --healthcheck`.

8.2. Proxy Inverso (Coolify / Traefik)

Para que ASP.NET Core conozca la IP real del cliente y sepa que la conexión original era HTTPS:

```
csharp
builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders = ForwardedHeaders.XForwardedFor |
ForwardedHeaders.XForwardedProto;
    options.KnownProxies.Add(IPAddress.Parse(proxyIp));
});
```

Se configuran proxies de confianza explícitos (`Security:TrustedProxies`), evitando envenenamiento de cabeceras HTTP.

8.3. Migraciones y Siembra Seguras

Payé Base **nunca ejecuta `Database.Migrate()` automáticamente al arrancar en producción**, para evitar carreras en despliegues con réplicas o bloqueos accidentales. Las migraciones y la siembra inicial se ejecutan mediante comandos CLI controlados:

```
bash
dotnet Paye.Web.dll --migrate
dotnet Paye.Web.dll --seed
```

8.4. Pruebas Automatizadas con `TestDatabaseGuard`

Las pruebas de integración en `tests/Paye.Tests` levantan una instancia real de PostgreSQL en un contenedor efímero con Testcontainers. Para garantizar que **nunca jamás** se ejecuten pruebas contra

la base de datos de producción o desarrollo, `TestDatabaseGuard` intercepta el inicio de las pruebas: si detecta que la cadena de conexión de prueba coincide con la de producción o no contiene los marcadores de aislamiento, **aborta la ejecución de tests en el acto**.

9. Comparativa Rápida: Paradigmas Antiguos vs. Payé Base

Aspecto	ASP.NET de los 90/2000	SPAs Desacopladas Frágiles	**Payé Base (.NET 10 LTS)**
Alojamiento	Exclusivo de Windows / IIS	Node.js + Backend disperso	Contenedor Docker Linux ultraligero (Kestrel)
Gestión de Estado	`ViewState` pesado en HTML	Tokens JWT en `localStorage` (XSS)	Cookies `HttpOnly` + `Secure` + Data Protection API
Seguridad de Rutas	Manual por página `.aspx`	Guardas en cliente (fácil de saltar)	`Deny-by-default` global (`FallbackPolicy` en servidor)
Autorización	Nombres de rol fijos en código	Booleans en payload de token	Permisos dinámicos por módulo (`Claims` en BD)
Interficie de Usuario	Recarga completa de página	Carga lenta, SEO roto, JWTs inseguros	Puente híbrido: Layout seguro + React SPA en `#root`
Base de Datos	Procedimientos almacenados o ADO	ORMs sin control de concurrencia	EF Core 10 + Concurrencia `xmin` + Soft-delete + Auditoría
Modo Demo	`admin/admin123` hardcodeado	Bypass de auth en frontend	Sandbox efímero sin contraseña y purga automática

10. Conclusión

Payé Base no es una simple colección de archivos; es un sistema coherente donde la seguridad, el rendimiento y la mantenibilidad están sellados en la propia arquitectura. Al aprovechar al 100% las capacidades nativas de **ASP.NET Core 10 LTS**, elimina la deuda técnica antes de que nazca, proporcionando una base indestructible para cualquier aplicación comercial a medida.